NORMALIZATION

# One Example, Every Normal Form

A single running scenario — students, courses, and instructors — carried step by step from an unnormalized table through 1NF, 2NF, 3NF, BCNF, 4NF, and 5NF.

*A University Enrollment System, decomposed one anomaly at a time*

| Stu_ID | Course |
| --- | --- |
| — | — |
| — | — |
| — | — |
| — | — |

THE SETUP

# One messy table to start

*Everything about a student's enrollment, jammed into a single relation*

| Stu_ID | Stu_Name | Stu_Phone | Course_ID | Course_Name | Fee | Instructor | Instructor_Phone |
|---|---|---|---|---|---|---|---|
| 1 | Asha | 99988-87771, 99988-87772 | C101 | DBMS | 6000 | Mehta | 88877-66610 |
| 1 | Asha | 99988-87771, 99988-87772 | C102 | Networks | 5500 | Rao | 88877-66620 |
| 2 | Rahul | 99977-76660 | C101 | DBMS | 6000 | Mehta | 88877-66610 |

*StudentEnrollment (unnormalized)*

- **Stu_Phone holds multiple values** Asha's two numbers are stuffed into one cell — not atomic.
- **Stu_Name repeats** "Asha" is written out again for every course she takes.
- **Course details repeat** DBMS's fee and instructor are copied for every student enrolled in it.
- **Update risk** Change Mehta's phone number and you must find and edit every row that mentions him.

1NF

# First Normal Form

*Rule: every attribute must hold a single, atomic value — no repeating groups*

**Violation: Stu_Phone stores two numbers in one cell**

| Stu_ID | Stu_Name | Stu_Phone |
|---|---|---|
| 1 | Asha | 99988-87771, 99988-87772 |
| 2 | Rahul | 99977-76660 |



**Fix: split the multi-valued cell into one row per value**

| Stu_ID | Stu_Name | Stu_Phone |
|---|---|---|
| 1 | Asha | 99988-87771 |
| 1 | Asha | 99988-87772 |
| 2 | Rahul | 99977-76660 |

## Why it matters

Databases can only compare, index, and filter atomic values. A cell holding “99988-87771, 99988-87772” can't be searched for a single phone number. 1NF trades a shorter table for one where every value is queryable on its own.

**In our example:**

Asha's row splits in two — one per phone number — so Stu_ID and Stu_Name repeat, but every field is now a single value. This repetition is exactly what the next normal forms clean up.

2NF

# Second Normal Form

*Rule: be in 1NF, and every non-key attribute must depend on the WHOLE primary key*

**Key: (Stu_ID, Course_ID)**

| Stu_ID | Course_ID | Stu_Name | Course_Name | Fee |
|---|---|---|---|---|
| 1 | C101 | Asha | DBMS | 6000 |
| 1 | C102 | Asha | Networks | 5500 |
| 2 | C101 | Rahul | DBMS | 6000 |

**Partial dependency:** Stu_Name depends only on Stu_ID, and Course_Name/Fee depend only on Course_ID — not on the full composite key.



**Fix: split off each partial dependency into its own table**

**Student**

| Stu_ID | Stu_Name |
|---|---|
| 1 | Asha |
| 2 | Rahul |

**Course**

| Course_ID | Course_Name | Fee |
|---|---|---|
| C101 | DBMS | 6000 |
| C102 | Networks | 5500 |

**Enrollment**

| Stu_ID | Course_ID |
|---|---|
| 1 | C101 |
| 1 | C102 |
| 2 | C101 |

## Why it matters

With a composite key, some columns only need part of it. Splitting them out means Asha's name is stored once, not once per course — and updating it never touches the Enrollment table at all.

3NF

# Third Normal Form

*Rule: be in 2NF, and no non-key attribute may depend on another non-key attribute*

Key: Course_ID

| Course_ID | Course_Name | Fee | Instructor | Instructor_Phone |
|---|---|---|---|---|
| C101 | DBMS | 6000 | Mehta | 88877-66610 |
| C102 | Networks | 5500 | Rao | 88877-66620 |

**Transitive dependency:** Course_ID ⟶ Instructor ⟶ Instructor_Phone. The phone number depends on the instructor, not directly on the course.

**Fix: move the transitively dependent attribute to its own table**

**Course**

| Course_ID | Course_Name | Fee | Instructor |
|---|---|---|---|
| C101 | DBMS | 6000 | Mehta |
| C102 | Networks | 5500 | Rao |

**Instructor**

| Instructor | Instructor_Phone |
|---|---|
| Mehta | 88877-66610 |
| Rao | 88877-66620 |

## Why it matters

Before this split, Mehta's phone number is copied onto every course he teaches. Storing it once in an Instructor table means one update, one source of truth, no risk of the copies disagreeing.

BCNF

# Boyce–Codd Normal Form

*Rule: for every dependency X → Y, X must be a super key — a stricter 3NF*

Scenario: an instructor teaches only one course, but a course can have several instructors

| Student | Course | Instructor |
|---|---|---|
| Asha | DBMS | Mehta |
| Rahul | DBMS | Mehta |
| Asha | Networks | Rao |

**Violation:** Instructor → Course holds (Mehta always means DBMS), but Instructor is not a super key here — (Student, Instructor) is.

**Fix: split so the determinant (Instructor) becomes a key**

**Student–Instructor**

| Student | Instructor |
|---|---|
| Asha | Mehta |
| Rahul | Mehta |
| Asha | Rao |

**Instructor–Course**

| Instructor | Course |
|---|---|
| Mehta | DBMS |
| Rao | Networks |

## Why it matters

3NF only checks dependencies where the determinant isn't a key AND the dependent isn't part of a key. BCNF closes that loophole: any attribute that determines another must itself be a full key — here, Instructor determines Course, so Instructor needs its own table.

4NF

# Fourth Normal Form

*Rule: be in BCNF, with no multi-valued dependency between independent attributes*

Scenario: a student's courses and hobbies are recorded together, but they have nothing to do with each other

| Student | Course | Hobby |
|---|---|---|
| Asha | DBMS | Chess |
| Asha | Networks | Painting |
| Rahul | DBMS | Football |

**Violation:** Student ⟶⟶ Course and Student ⟶⟶ Hobby (Asha's courses and hobbies) are independent multi-valued facts, forced into a cross-product of rows.



**Fix: separate each independent multi-valued fact**

**Student–Course**

| Student | Course |
|---|---|
| Asha | DBMS |
| Asha | Networks |
| Rahul | DBMS |

**Student–Hobby**

| Student | Hobby |
|---|---|
| Asha | Chess |
| Asha | Painting |
| Rahul | Football |

## Why it matters

Because the table lists every course × hobby combination for a student, adding one new hobby for Asha would mean adding a new row for every course she's taking — pure redundancy with no new information. Splitting course-taking and hobby-having into separate tables removes that forced pairing entirely.

5NF

# Fifth Normal Form

*Rule: be in 4NF, with no join dependency — splitting further must stay lossless*

Scenario: instructor, course, and semester only make sense together as specific triples

| Instructor | Course | Semester |
|---|---|---|
| Mehta | DBMS | Sem 1 |
| Mehta | Networks | Sem 2 |
| Rao | DBMS | Sem 1 |

**Join dependency:** Mehta teaches DBMS, and Mehta teaches in Sem 1 — but that does NOT mean Mehta teaches DBMS in Sem 1 unless the actual row says so. Splitting into any two pairwise tables and rejoining would reintroduce a row that never existed.

**Fix: decompose into all three pairwise relations — only their common join recreates exactly the valid rows**

**Instructor–Course**

| Instructor | Course |
|---|---|
| Mehta | DBMS |
| Mehta | Networks |
| Rao | DBMS |

**Course–Semester**

| Course | Semester |
|---|---|
| DBMS | Sem 1 |
| Networks | Sem 2 |

**Instructor–Semester**

| Instructor | Semester |
|---|---|
| Mehta | Sem 1 |
| Mehta | Sem 2 |
| Rao | Sem 1 |

## Why it matters

5NF (Project-Join Normal Form) is about the rare case where no two-way split is lossless, only a three-way one is. It's mostly a theoretical ceiling — real schemas rarely need to be pushed this far.

RECAP

# The whole journey, at a glance

*Same student, same courses, same instructors — one fix per normal form*

| Form | Rule | What we fixed |
|---|---|---|
| 1NF | Atomic values only | Split Asha's two phone numbers into separate rows |
| 2NF | No partial dependency on the key | Pulled Stu_Name and Course details out of the Enrollment table |
| 3NF | No transitive dependency | Moved Instructor_Phone into its own Instructor table |
| BCNF | Every determinant is a super key | Split Instructor → Course out from Student–Instructor–Course |
| 4NF | No independent multi-valued facts | Separated Student–Course from Student–Hobby |
| 5NF | No join dependency | Decomposed Instructor–Course–Semester into three pairwise tables |

# The pattern behind every step

Each normal form asks the same question in a stricter way: “does this attribute truly belong here, or does it depend on something else entirely?” Every time the answer was “something else,” we gave that fact its own table — trading a single wide relation for several narrow, honest ones.

*Less redundancy. Fewer anomalies. One source of truth per fact.*